รายวิชา IT368

Cloud Computing

Lecture 7

Kubernetes Fundamentals

พื้นฐาน Kubernetes

เข้าใจแนวคิด เรียนรู้การใช้งาน และการจัดการคอนเทนเนอร์
เพื่อรองรับแอปพลิเคชันบนคลาวด์



Understand Concepts | Hands-on Practice | Real-world Skills | Prepare for the Cloud

จาก Docker สู่ Kubernetes

ในบทที่แล้ว เราเรียนรู้การสร้างและรัน Container ด้วย Docker ซึ่งเหมาะกับการใช้งานแอปพลิเคชันขนาดเล็ก หรือมีจำนวนไม่มาก แต่เมื่อแอปพลิเคชันมีหลายส่วน และต้องรันหลาย Container เราต้องการเครื่องมือที่ช่วยจัดการให้เป็นระบบมากขึ้น นั่นคือ Kubernetes

Docker

จัดการ Container ทีละตัว

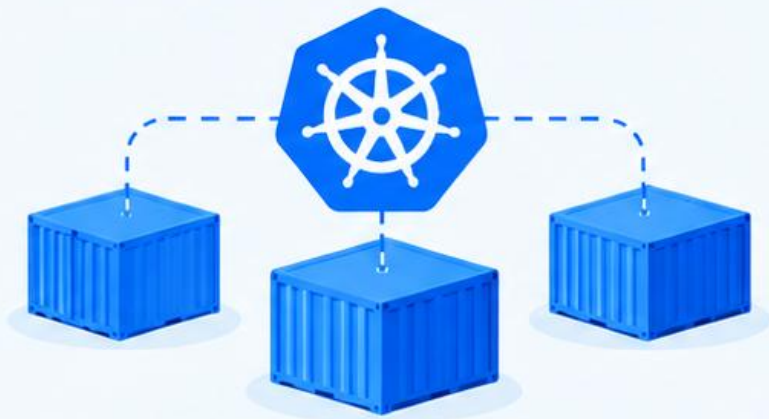


- สร้างและรัน Container ได้ง่าย
- เหมาะกับแอปพลิเคชันขนาดเล็ก
- จัดการได้ดีเมื่อมีจำนวนไม่มาก

จาก 1 Container
สู่หลาย Container

Kubernetes

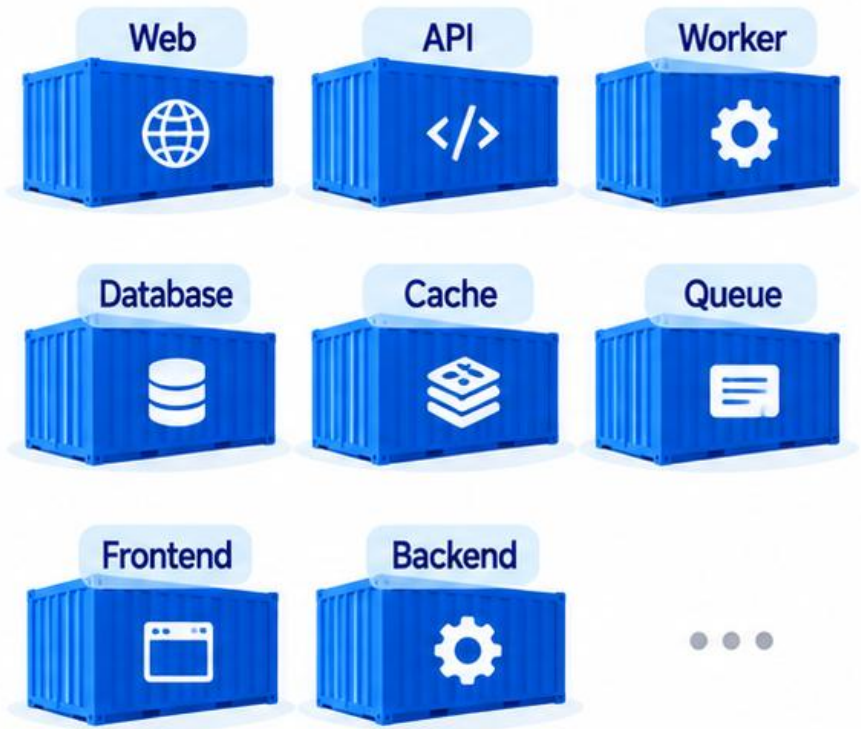
จัดการหลาย Container แบบอัตโนมัติ



- จัดการ Container จำนวนมาก
- รองรับการขยายระบบ (Scaling)
- ดูแลความพร้อมใช้งาน (Self-healing)

ปัญหาของการจัดการ Container จำนวนมาก

เมื่อแอปพลิเคชันมีหลายส่วน (เช่น เว็บเซิร์ฟเวอร์, ฐานข้อมูล, แคช, คิวงาน ฯลฯ) เราต้องรันหลาย Container และต้องดูแลให้ทำงานร่วมกันตลอดเวลา การใช้คำสั่ง Docker เพียงอย่างเดียวอาจไม่เพียงพอ



การติดตั้งและเริ่มต้น
ต้องรันหลาย Container และกำหนดการเชื่อมต่อกัน

การเชื่อมต่อและเครือข่าย
แต่ละ Container ต้องสื่อสารกัน เช่น Web → API → Database

การขยายระบบ (Scaling)
เมื่อมีผู้ใช้งานเพิ่ม ต้องเพิ่มจำนวน Container ได้อย่างรวดเร็ว

ความพร้อมใช้งาน
หาก Container ล่ม, ต้องเริ่มใหม่โดยอัตโนมัติ

การอัปเดตและดูแลรักษา
ต้องอัปเดตเวอร์ชันใหม่โดยไม่หยุดบริการ และจัดการหลาย Container พร้อมกัน

หากจัดการด้วย Docker เพียงอย่างเดียว

- ต้องใช้คำสั่งจำนวนมาก
- ยากต่อการดูแลเมื่อมีหลาย Container
- ไม่มีการเริ่มใหม่อัตโนมัติเมื่อ Container ล่ม
- ไม่มีการขยายระบบอัตโนมัติ
- ยากต่อการอัปเดตแบบไม่หยุดบริการ
- ไม่เหมาะกับระบบที่ต้องการความพร้อมใช้งานสูง

Container Orchestration คืออะไร?

Container Orchestration คือการจัดการ Container จำนวนมากแบบอัตโนมัติ ทำหน้าที่จัดสรรทรัพยากร กำหนดการรัน ปรับขยาย และดูแลให้แอปพลิเคชันทำงานได้อย่างต่อเนื่อง

จัดการด้วย Docker (ทีละเครื่อง)

เหมาะสำหรับแอปขนาดเล็ก หรือจำนวน Container ไม่มาก

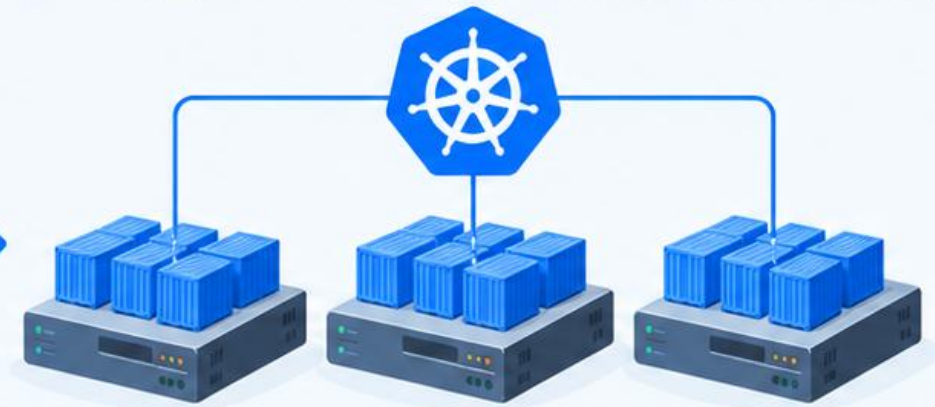


- ✅ สร้างและรัน Container ได้ง่าย
- ✅ เหมาะกับงานขนาดเล็ก หรือทดสอบ
- ❌ ต้องจัดการเองทั้งหมด
- ❌ ไม่เหมาะกับการรันหลายเครื่อง หรือหลาย Container

เมื่อมี Container มากขึ้น และต้องรันบนหลายเครื่อง

ใช้ Container Orchestration (เช่น Kubernetes)

จัดการ Container จำนวนมากแบบอัตโนมัติ บนหลายเครื่อง (หลายโหนด)



- ✅ จัดสรรและรัน Container บนหลายเครื่อง (หลายโหนด)
- ✅ ปรับขยายอัตโนมัติ (Scaling)
- ✅ ดูแลความพร้อมใช้งาน (Self-healing)
- ✅ บริหารจัดการแบบรวมศูนย์
- ✅ เหมาะกับระบบขนาดใหญ่ในสภาพแวดล้อมคลาวด์

Kubernetes คืออะไร?

Kubernetes (K8s) เป็นระบบจัดการ Container แบบโอเพนซอร์ส ช่วยให้เราสามารถรัน ปรับขยาย และดูแลแอปพลิเคชันที่ประกอบด้วยหลาย Container ได้อย่างอัตโนมัติบนหลายเครื่องในรูปแบบคลัสเตอร์ (Cluster)

จัดการ Container จำนวนมาก

รันหลาย Container หลายแอปพลิเคชันบนหลายเครื่องได้อย่างมีประสิทธิภาพ

ทำงานอัตโนมัติ

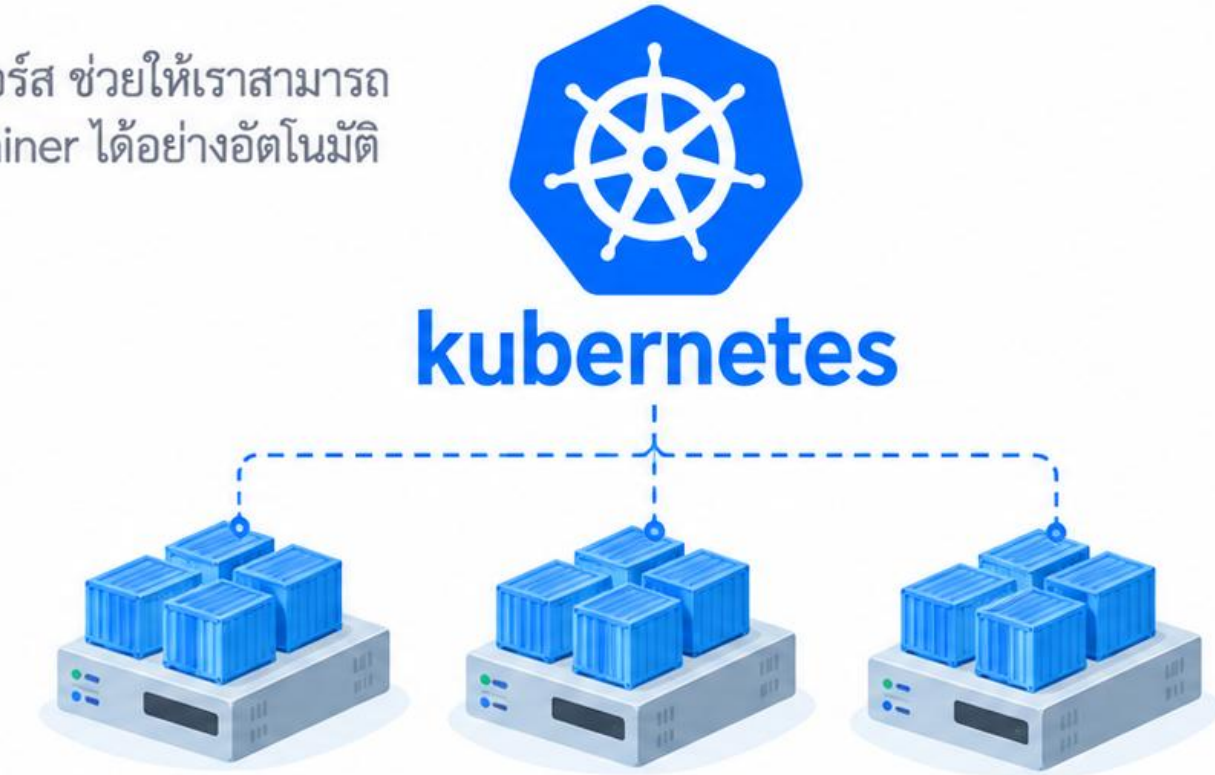
จัดสรรทรัพยากร ปรับขยาย และดูแลการทำงานโดยอัตโนมัติ

มีความทนทานและพร้อมใช้งานสูง

ตรวจสอบสถานะและกู้คืนเมื่อเกิดความล้มเหลวของ Container หรือเครื่อง

รองรับการใช้งานบน Cloud

สามารถใช้งานได้ทั้งบนคลาวด์ (เช่น GKE, EKS, AKS) หรือบนโครงสร้างพื้นฐานขององค์กรเอง



Scaling
เพิ่ม/ลดจำนวน Container ตามการใช้งานได้อัตโนมัติ

Self-healing
ตรวจจับปัญหาและสร้าง Container ใหม่อัตโนมัติ

High Availability
กระจายการทำงานบนหลายเครื่อง ลดความเสี่ยง

Kubernetes ช่วย Cloud Application อย่างไร?

Kubernetes ช่วยให้การพัฒนาและใช้งานแอปพลิเคชันบนคลาวด์ทำได้ง่ายขึ้น มีความยืดหยุ่น ปลอดภัย และพร้อมรองรับการใช้งานในระดับขนาดใหญ่

ปรับขยายอัตโนมัติ (Auto Scaling)

เพิ่มหรือลดจำนวน Container ตามปริมาณการใช้งานได้โดยอัตโนมัติ



รองรับการใช้งานตลอดเวลา

หาก Container หรือเครื่องล่ม Kubernetes จะสร้างใหม่ให้อัตโนมัติ ทำให้แอปพลิเคชันยังคงพร้อมใช้งาน



บริหารจัดการง่ายขึ้น

จัดการแอปพลิเคชันหลายตัวบนหลายเครื่องได้จากจุดเดียว ด้วยคำสั่งและไฟล์กำหนดค่าแบบเดียวกัน



ใช้ทรัพยากรได้คุ้มค่า

กระจาย Container ไปยังเครื่องที่มีทรัพยากรว่าง ช่วยให้ใช้ CPU และหน่วยความจำได้อย่างมีประสิทธิภาพ



ปลอดภัยและเชื่อถือได้

มีระบบควบคุมการเข้าถึง จัดการความปลอดภัยของแอปพลิเคชัน และแยกแอปพลิเคชันให้ทำงานอย่างปลอดภัย



รองรับการใช้งานบน Cloud

สามารถใช้งานได้ทั้งบนคลาวด์สาธารณะ (เช่น GKE, EKS, AKS) หรือบนโครงสร้างพื้นฐานขององค์กรเอง



Docker vs Kubernetes ต่างกันอย่างไร?

Docker และ Kubernetes ทำงานร่วมกันได้ โดย Docker ใช้สร้างและรัน Container ส่วน Kubernetes ใช้จัดการ Container จำนวนมากแบบอัตโนมัติบนหลายเครื่อง เหมาะกับการใช้งานระดับใหญ่

Docker

เครื่องมือสำหรับสร้างและรัน Container



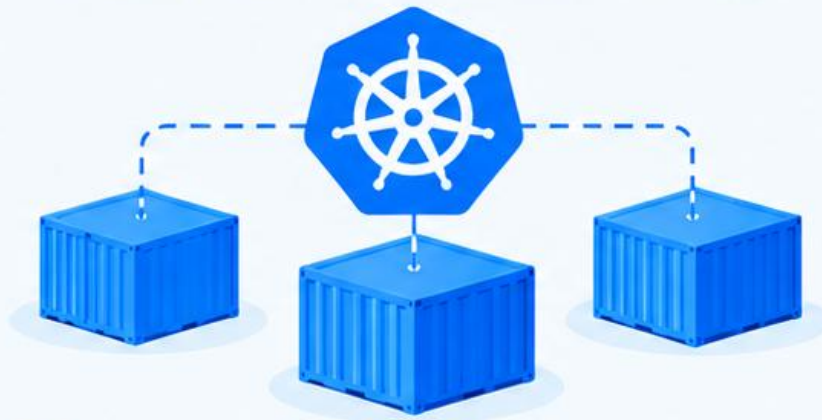
- สร้างและแพ็กแอปพลิเคชันเป็น Container
- รันและทดสอบ Container บนเครื่องเดียว
- ใช้งานง่าย เหมาะสำหรับพัฒนาและทดสอบ
- จัดการ Container ทีละตัว

ทำงานร่วมกันได้

ใช้ Docker สร้าง Container และใช้ Kubernetes จัดการ Container จำนวนมาก

Kubernetes

ระบบจัดการ Container จำนวนมากแบบอัตโนมัติ



- จัดการ Container หลายตัวบนหลายเครื่อง
- ปรับขยายอัตโนมัติ (Scaling)
- ดูแลความพร้อมใช้งาน (Self-healing)
- เหมาะกับระบบขนาดใหญ่ในสภาพแวดล้อมคลาวด์

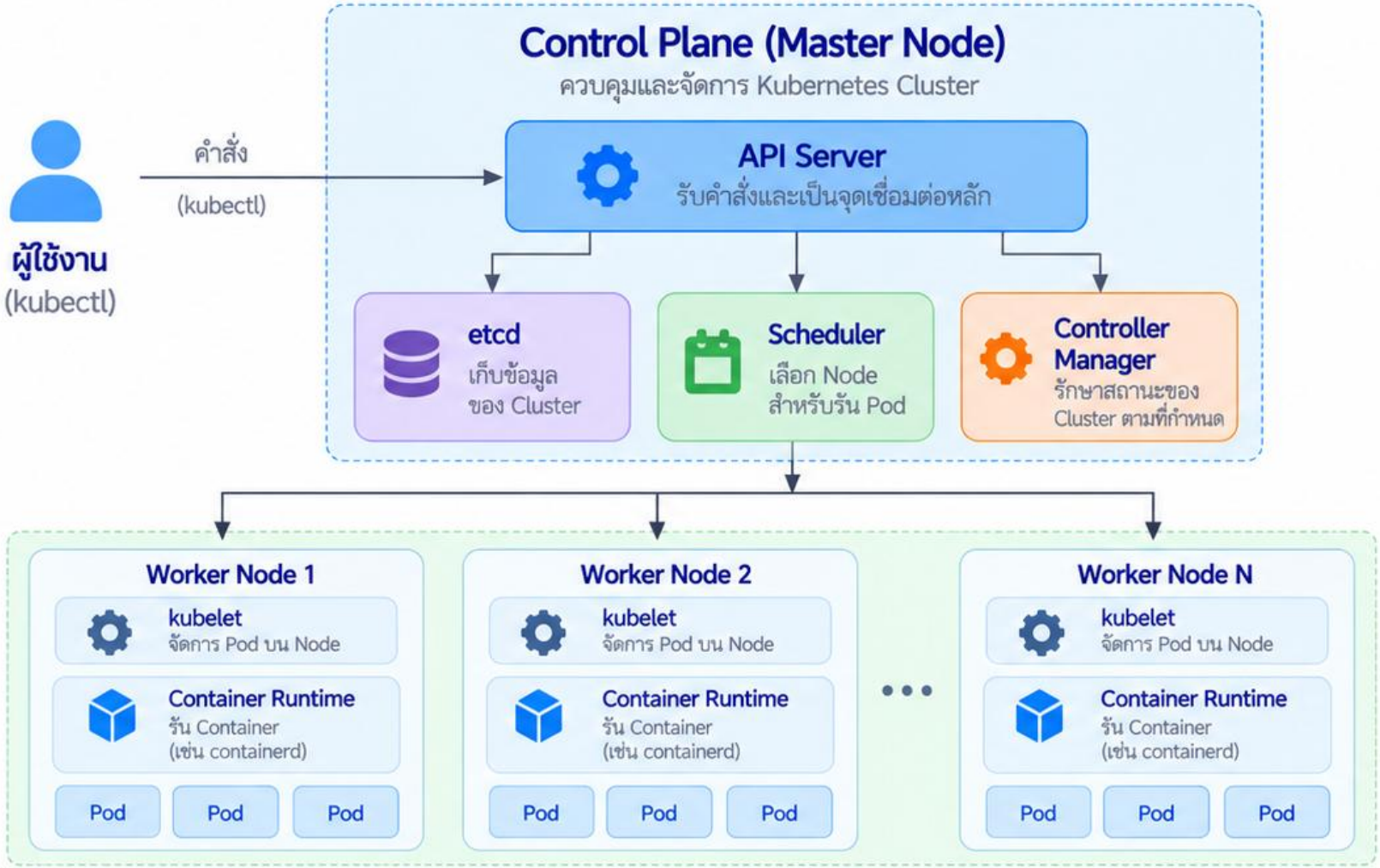
เปรียบเทียบ Docker กับ Kubernetes

Docker และ Kubernetes มีบทบาทที่ต่างกัน โดย Docker เน้นการสร้างและรัน Container ส่วน Kubernetes เน้นการจัดการ Container จำนวนมากแบบอัตโนมัติ

ประเด็น	Docker	Kubernetes
วัตถุประสงค์	สร้างและรัน Container	จัดการ Container จำนวนมากแบบอัตโนมัติ
ระดับการใช้งาน	เครื่องเดียว (Single Host)	หลายเครื่อง (Cluster)
สิ่งที่จัดการ	Container แต่ละตัว	หลาย Container, โหลดบาลานซ์, Scaling, การกู้คืนอัตโนมัติ ฯลฯ
ความซับซ้อน	ใช้งานง่าย เรียนรู้ได้เร็ว	ซับซ้อนมากกว่า ต้องเรียนรู้หลายองค์ประกอบ
เหมาะกับงานแบบไหน	พัฒนาและทดสอบแอปพลิเคชันขนาดเล็กถึงขนาดกลาง	ระบบขนาดใหญ่ที่ต้องการความพร้อมใช้งานสูงและการขยายตัวอัตโนมัติ
ตัวอย่างการใช้งาน	รันเว็บแอป, API, ฐานข้อมูล บนเครื่องเดียว	จัดการเว็บแอป, API, ฐานข้อมูล หลายตัวบนหลายเครื่องในคลาวด์หรือดาต้าเซ็นเตอร์

สถาปัตยกรรมของ Kubernetes

Kubernetes เป็นระบบแบบ Master-Worker (Control Plane - Worker Node) โดยมี Component ต่าง ๆ ทำงานร่วมกันเพื่อจัดการ Container และรักษาสถานะของระบบให้ทำงานได้อย่างต่อเนื่อง



หน้าที่ของ Component สำคัญ

API Server
รับคำสั่งจากผู้ใช้งาน (kubectl) และเป็นจุดเชื่อมต่อระหว่าง Component ต่าง ๆ

etcd
ฐานข้อมูลแบบ Key-Value สำหรับเก็บข้อมูลของ Cluster

Scheduler
เลือก Worker Node ที่เหมาะสมสำหรับรัน Pod ตามทรัพยากรและนโยบาย

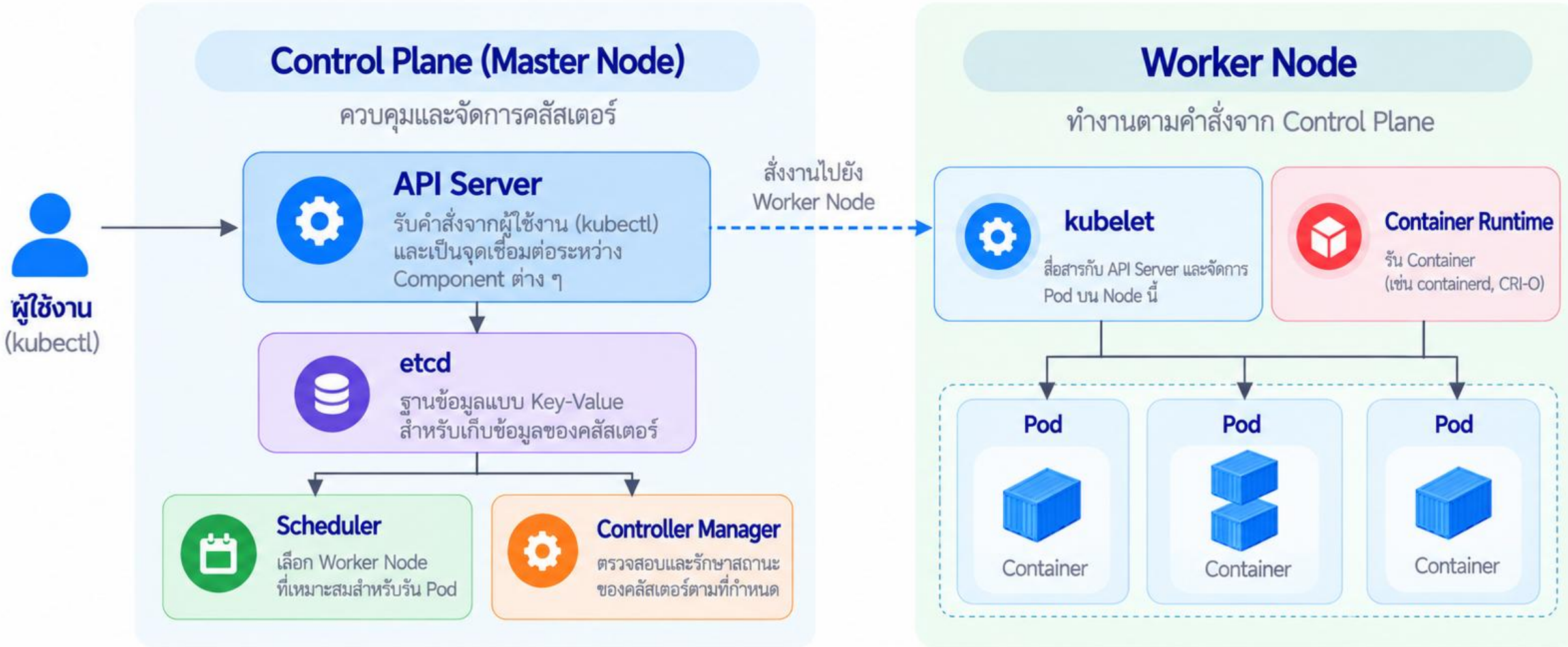
Controller Manager
ตรวจสอบและรักษาสถานะของระบบให้ตรงกับที่กำหนด (เช่น จำนวน Pod)

kubelet (บน Worker Node)
สื่อสารกับ API Server และจัดการ Pod บน Node นั้น

Container Runtime (บน Worker Node)
ทำหน้าที่รัน Container (เช่น containerd, CRI-O)

องค์ประกอบหลักของ Kubernetes

Kubernetes ประกอบด้วย Control Plane และ Worker Node โดยแต่ละส่วนมีหน้าที่แตกต่างกัน และทำงานร่วมกัน เพื่อจัดการ Container ในคลัสเตอร์



รายละเอียด Component หลักของ Kubernetes

Kubernetes ประกอบด้วย Component หลักที่ทำงานร่วมกัน เพื่อจัดการ Container และรักษาสถานะของระบบ

Control Plane (Master Node)

ควบคุมและจัดการทั้งคลัสเตอร์



API Server

จุดเชื่อมต่อหลักของคลัสเตอร์

- รับคำสั่งจากผู้ใช้งาน (kubectl)
- ตรวจสอบและประมวลผลคำสั่ง
- สื่อสารกับ Component อื่น ๆ



etcd

ฐานข้อมูลของคลัสเตอร์

- เก็บข้อมูลแบบ Key-Value
- เช่น ข้อมูลของ Pod, Service, Deployment, Config
- มีความสำคัญต่อความถูกต้องและความเสถียรของคลัสเตอร์



Scheduler

เลือก Node สำหรับรัน Pod

- พิจารณาทรัพยากรของแต่ละ Node
- เลือก Node ที่เหมาะสมสำหรับ Pod
- คำนึงถึงข้อกำหนด เช่น CPU, Memory, Affinity, Taints/Tolerations



Controller Manager

ดูแลให้ระบบเป็นไปตามสถานะที่กำหนด

- ตรวจสอบสถานะของ Object ต่าง ๆ
- สร้าง/ลบ/แทนที่ Pod ตามที่กำหนด
- เช่น ReplicaSet, Deployment, Node Controller, Endpoint Controller

Worker Node

ทำงานตามคำสั่งจาก Control Plane เพื่อรัน Container



kubelet

ตัวแทนบน Node

- รับคำสั่งจาก API Server
- จัดการ Pod บน Node นั้น
- ตรวจสอบและรายงานสถานะของ Pod



Container Runtime

รัน Container

- ทำหน้าที่รัน Container จริง
- เช่น containerd, CRI-O
- ดาวน์โหลด Image, สร้าง และหยุด Container ตามคำสั่งของ kubelet



kube-proxy

จัดการเครือข่าย

- จัดการกฎการส่งต่อเครือข่าย (iptables/IPVS)
- ทำให้ Service สามารถเข้าถึง Pod ได้
- กระจายทราฟฟิกไปยัง Pod ที่เหมาะสม



Pod

หน่วยที่รัน Container

- กลุ่มของ Container ที่ทำงานร่วมกัน
- มี IP Address เดียวกัน
- เป็นหน่วยพื้นฐานที่ Kubernetes จัดการ

Pod คืออะไร?

Pod เป็นหน่วยพื้นฐานที่เล็กที่สุดของ Kubernetes ใช้สำหรับรัน Container หนึ่งตัวหรือหลายตัว ที่ทำงานร่วมกัน โดยแชร์เครือข่ายและพื้นที่จัดเก็บข้อมูลเดียวกัน

ลักษณะสำคัญของ Pod

หน่วยพื้นฐาน
เป็นหน่วยที่เล็กที่สุดใน Kubernetes

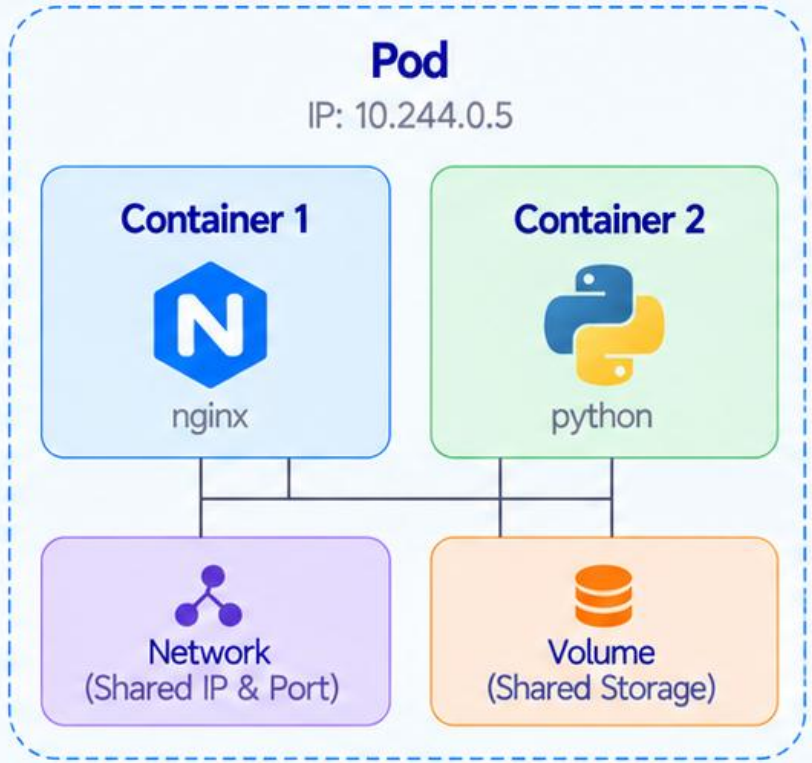
รันได้หลาย Container
มักรัน 1 Container และบางกรณี รันหลาย Container ที่ต้องทำงานร่วมกัน

แชร์เครือข่ายเดียวกัน
Container ภายใน Pod ใช้ IP Address และ Port เดียวกัน

แชร์พื้นที่จัดเก็บข้อมูล
สามารถใช้ Volume เดียวกัน ในการแชร์ไฟล์ระหว่าง Container

จัดการแบบกลุ่ม
Pod จะถูกสร้าง ลบ และรีสตาร์ต พร้อมกันทั้ง Pod

โครงสร้างของ Pod



ตัวอย่าง YAML สร้าง Pod

pod.yaml

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: web-pod
spec:
  containers:
    - name: nginx
      image: nginx:1.25
      ports:
        - containerPort: 80

    - name: app
      image: python:3.11-slim
      command: ["python", "-m", "http.server"]
      ports:
        - containerPort: 8000
      volumeMounts:
        - name: shared-data
          mountPath: /data

  volumes:
    - name: shared-data
      emptyDir: {}
```

Pod เหมาะสำหรับรัน Container ที่ทำงานร่วมกันและต้องการแชร์เครือข่ายหรือข้อมูล เช่น เว็บเซิร์ฟเวอร์กับแอปพลิเคชัน

Deployment คืออะไร?

Deployment เป็น Resource ของ Kubernetes ที่ใช้สำหรับจัดการการรันและอัปเดต Pod โดยอัตโนมัติ
ช่วยให้แอปพลิเคชันทำงานได้อย่างต่อเนื่องและรองรับการขยายจำนวน Pod

หน้าที่ของ Deployment

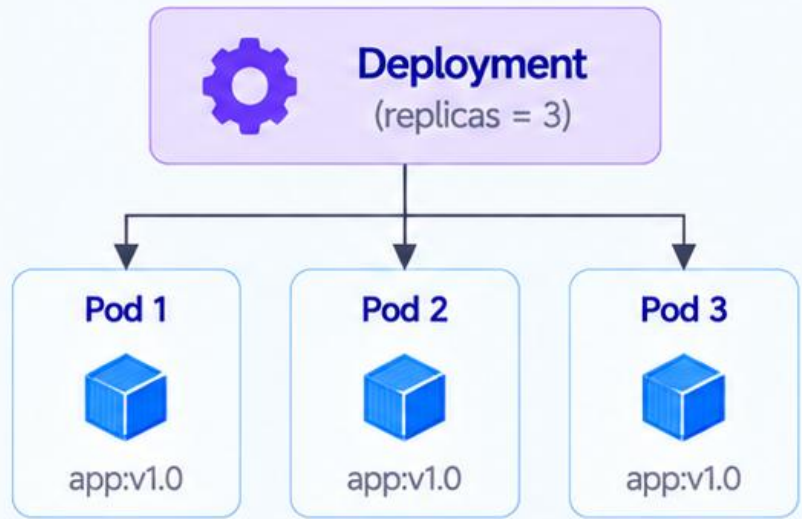
รันและรักษาจำนวน Pod
สร้าง Pod ตามจำนวนที่กำหนด
และตรวจสอบให้ทำงานอยู่เสมอ

รองรับการอัปเดตแบบ Rolling Update
อัปเดตเวอร์ชันของแอปพลิเคชัน
โดยไม่หยุดการทำงานทั้งหมด

รองรับการขยาย (Scaling)
เพิ่มหรือลดจำนวน Pod ได้ง่าย

Self-healing
หาก Pod ล้มเหลว จะสร้าง Pod ใหม่
ขึ้นมาโดยอัตโนมัติ

การทำงานของ Deployment



หาก Pod ใดล้มเหลว
Deployment จะสร้าง Pod ใหม่ให้โดยอัตโนมัติ

ตัวอย่าง YAML สร้าง Deployment

deployment.yaml

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web-app
spec:
  replicas: 3
  selector:
    matchLabels:
      app: web-app
  template:
    metadata:
      labels:
        app: web-app
    spec:
      containers:
      - name: nginx
        image: nginx:1.25
        ports:
          - containerPort: 80
```

Deployment เหมาะสำหรับแอปพลิเคชันที่ต้องการความพร้อมใช้งานสูง และต้องการอัปเดตหรือขยายจำนวน Pod ได้ง่าย

Service คืออะไร?

Service เป็น Resource ของ Kubernetes ที่ใช้เปิดการเข้าถึง (Networking) ไปยัง Pod หรือกลุ่มของ Pod โดยให้ IP และ DNS ที่คงที่ และสามารถทำ Load Balancing ไปยัง Pod หลายตัวได้

ทำไมต้องใช้ Service?

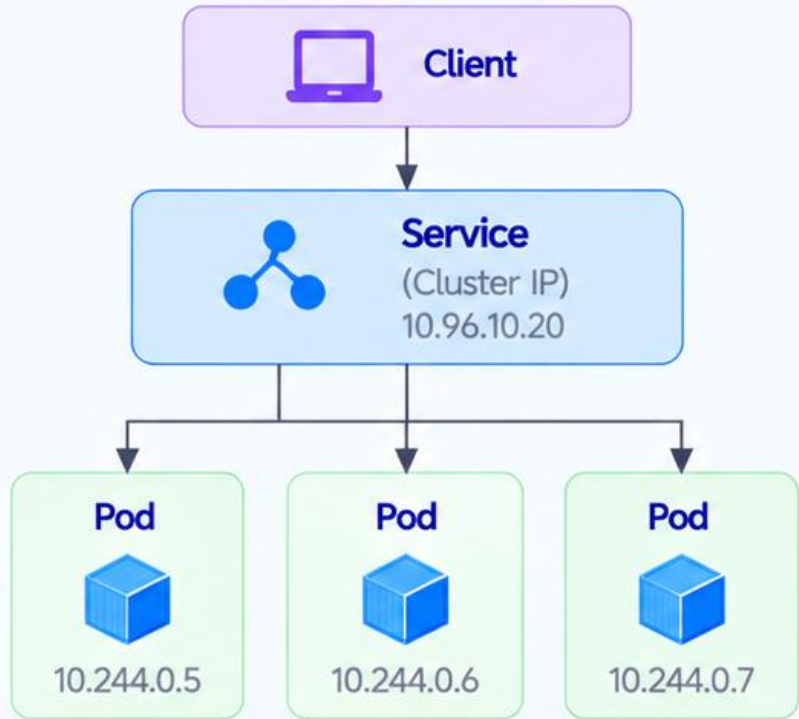
ให้ที่อยู่ที่คงที่
Pod อาจถูกสร้าง/ลบ ทำให้ IP เปลี่ยน เสมอ Service จึงให้ IP และ DNS ที่คงที่

ทำ Load Balancing
กระจายการเชื่อมต่อไปยัง Pod หลายตัวโดยอัตโนมัติ

เปิดการเข้าถึงจากภายใน/ภายนอก
รองรับหลายรูปแบบตามความต้องการ

แยกการเข้าถึงจากการจัดการ Pod
ไม่ต้องแก้ไขการเรียกใช้งาน แม้จำนวน Pod จะเปลี่ยน

การทำงานของ Service



ประเภทของ Service

ClusterIP (ค่าเริ่มต้น)	เข้าถึงได้เฉพาะภายใน Cluster ใช้สำหรับการสื่อสารระหว่าง Service ภายใน
NodePort	เปิดพอร์ตบนทุก Node ของ Cluster เพื่อให้เข้าถึงจากภายนอกได้ (เหมาะสำหรับการทดสอบ)
LoadBalancer	สร้าง Load Balancer จากผู้ให้บริการ Cloud เพื่อให้เข้าถึงจากภายนอก (เช่น AWS ELB, GCP LB, Azure LB)
ExternalName	เชื่อมต่อไปยังบริการภายนอก โดยใช้ชื่อ DNS

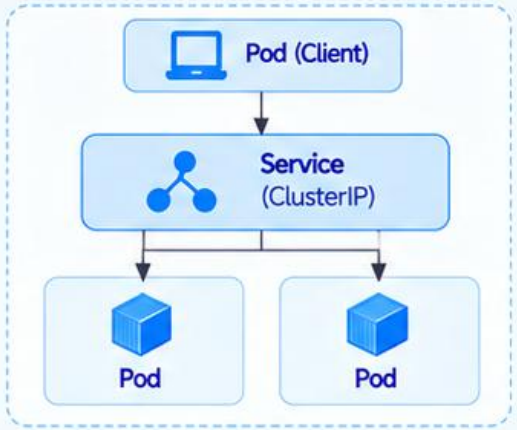
Service ทำหน้าที่เป็นตัวกลางในการเชื่อมต่อ ช่วยให้เข้าถึงแอปพลิเคชันได้อย่างเสถียร แม้จำนวน Pod จะเปลี่ยนแปลง

ประเภทของ Service

Kubernetes มี Service หลายประเภท เพื่อรองรับรูปแบบการเข้าถึงแอปพลิเคชันที่แตกต่างกัน

ClusterIP

ใช้ภายใน Cluster เท่านั้น



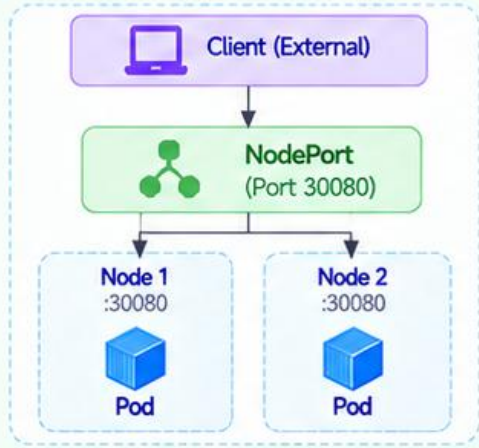
- สร้าง IP ภายใน Cluster
- เข้าถึงได้เฉพาะภายใน Cluster
- เหมาะสำหรับการสื่อสารระหว่าง Service ภายใน

service.yaml

```yaml
apiVersion: v1
kind: Service
metadata:
  name: web-svc
spec:
  type: ClusterIP
  selector:
    app: web
```

NodePort

เปิดพอร์ตบนทุก Node



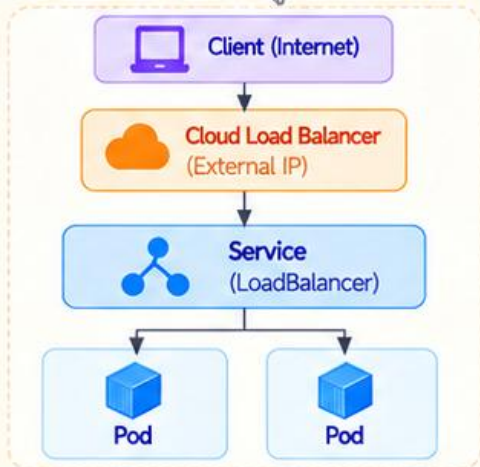
- เปิดพอร์ตเดียวกันบนทุก Node
- เข้าเข้าถึงจากภายนอกได้
- เหมาะสำหรับการทดสอบหรือใช้งานในสภาพแวดล้อมขนาดเล็ก

service.yaml

```yaml
apiVersion: v1
kind: Service
metadata:
  name: web-svc
spec:
  type: NodePort
  selector:
    app: web
  ports:
    - port: 80
      targetPort: 80
      nodePort: 30080
```

LoadBalancer

ใช้ Load Balancer จากผู้ให้บริการ Cloud



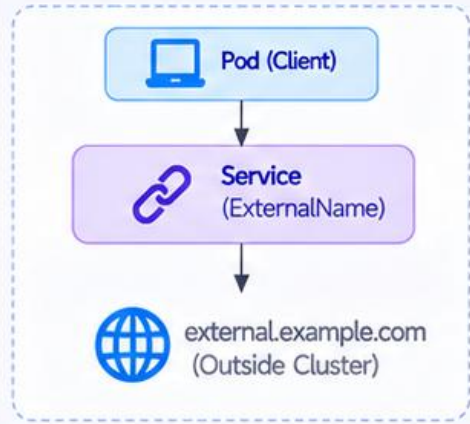
- สร้าง Load Balancer อัตโนมัติจาก Cloud
- ได้ IP สาธารณะ (External IP)
- เหมาะสำหรับให้ผู้ใช้งานภายนอกเข้าถึงแอปพลิเคชันจริง

service.yaml

```yaml
apiVersion: v1
kind: Service
metadata:
  name: web-svc
spec:
  type: LoadBalancer
  selector:
    app: web
  ports:
    - port: 80
      targetPort: 80
```

ExternalName

เชื่อมต่อไปยังบริการภายนอก



- ใช้ชื่อ DNS เพื่อเชื่อมต่อไปยังบริการภายนอก
- ไม่สร้าง Proxy หรือ Load Balancer
- เหมาะสำหรับเชื่อมต่อกับบริการที่อยู่นอก Kubernetes

service.yaml

```yaml
apiVersion: v1
kind: Service
metadata:
  name: external-svc
spec:
  type: ExternalName
  externalName: external.example.com
```

เลือกประเภทของ Service ให้เหมาะกับความต้องการ เช่น ใช้ ClusterIP ภายในระบบ, NodePort สำหรับทดสอบ, หรือ LoadBalancer สำหรับใช้งานจริงบน Cloud

การขยายและการฟื้นตัวอัตโนมัติ

Kubernetes สามารถปรับจำนวน Pod ตามโหลด (Scaling) และสร้าง Pod ขึ้นมาใหม่อัตโนมัติเมื่อเกิดปัญหา (Self-healing)

Scaling (เพิ่ม/ลดจำนวน Pod อัตโนมัติ)

ปรับจำนวน Pod ตามโหลดการใช้งาน เพื่อรองรับปริมาณผู้ใช้ที่เปลี่ยนแปลง



วิธีการ Scaling

Horizontal Pod Autoscaler (HPA)
เพิ่ม/ลดจำนวน Pod ตามค่า CPU, Memory หรือ Custom Metric

Manual Scaling
กำหนดจำนวน Pod เองด้วยคำสั่ง เช่น kubectl scale

Self-healing (การฟื้นตัวอัตโนมัติ)

หาก Pod ล้มเหลว Kubernetes จะสร้าง Pod ใหม่โดยอัตโนมัติ



กลไกที่เกี่ยวข้อง

ReplicaSet
รักษาจำนวน Pod ให้ตรงตามที่กำหนด หากมี Pod ล้มเหลว จะสร้างใหม่ทันที

Liveness / Readiness Probe
ตรวจสอบสุขภาพของแอปพลิเคชัน และรีสตาร์ต Pod หากตอบสนองไม่ได้

Scaling ช่วยให้ระบบรองรับการใช้งานที่เปลี่ยนแปลงได้อย่างยืดหยุ่น และ Self-healing ช่วยให้ระบบมีความเสถียรและพร้อมใช้งานตลอดเวลา

kubectl คืออะไร?

kubectl เป็น Command Line Tool สำหรับจัดการ Kubernetes Cluster
ใช้ในการสร้าง, ตรวจสอบ, อัปเดต, และลบ Resource ต่างๆ เช่น Pod, Deployment, Service

kubectl คืออะไร?

Command Line Tool
ใช้สั่งงานและจัดการ Kubernetes

จัดการ Resource
เช่น Pod, Deployment, Service, ConfigMap, Secret เป็นต้น

เชื่อมต่อกับ Cluster
ส่งคำสั่งไปยัง API Server ของ Kubernetes Cluster

รูปแบบคำสั่งพื้นฐาน

```
kubectl [command] [resource] [options]
```

คำสั่งที่ใช้บ่อย

`kubectl get`	ดูรายการ Resource
`kubectl describe`	ดูรายละเอียด Resource
`kubectl create`	สร้าง Resource จากไฟล์ YAML
`kubectl apply`	สร้างหรืออัปเดตจากไฟล์ YAML
`kubectl delete`	ลบ Resource
`kubectl logs`	ดู Log ของ Pod
`kubectl exec`	เข้าใช้งานภายใน Pod
`kubectl scale`	ปรับจำนวน Replica
`kubectl rollout`	จัดการการอัปเดตเวอร์ชัน

ตัวอย่างการใช้งาน

ดู Pod ทั้งหมด

```
$ kubectl get pods
```

```
NAME      READY   STATUS    RESTARTS   AGE
web-xyz   1/1     Running   0          2m
api-abc   1/1     Running   0          3m
```

ดูรายละเอียด Pod

```
$ kubectl describe pod web-xyz
```

```
Name:         web-xyz
Namespace:    default
Status:       Running
IP:           10.244.0.5
Containers:
  nginx:
    Image:    nginx:1.25
```

ดู Log ของ Pod

```
$ kubectl logs web-xyz
```

```
/docker-entrypoint.sh: Configuration complete
Nginx ready to serve requests...
```

kubectl เป็นเครื่องมือสำคัญที่ใช้ในการบริหารจัดการ Kubernetes ในชีวิตประจำวันของผู้ดูแลระบบและนักพัฒนา

ไฟล์ YAML ใน Kubernetes

Kubernetes ใช้ไฟล์ YAML ในการกำหนดรายละเอียดของ Resource เช่น Pod, Deployment, Service
โดยเราสามารถสร้าง แก้ไข และนำไฟล์ YAML ไปใช้งานด้วยคำสั่ง kubectl

YAML คืออะไร?

รูปแบบไฟล์แบบข้อความ
เขียนง่าย อ่านง่าย ใช้กำหนดค่าของ Resource ใน Kubernetes

กำหนดสถานะที่ต้องการ
บอก Kubernetes ว่าต้องการสร้าง Resource อะไร และมีคุณสมบัติอย่างไร

ใช้งานร่วมกับ kubectl
สร้าง แก้ไข ลบ และอัปเดต Resource ได้จากไฟล์ YAML

ใช้ได้กับหลาย Resource
เช่น Pod, Deployment, Service, ConfigMap, Secret เป็นต้น

โครงสร้างไฟล์ YAML (Deployment)

1. **apiVersion, kind**
   ระบุเวอร์ชันของ API และประเภทของ Resource
2. **metadata**
   กำหนดชื่อ และข้อมูลเพิ่มเติม
3. **spec**
   กำหนดรายละเอียดที่ต้องการ เช่น จำนวน Pod, Template ของ Pod
4. **template (Pod Template)**
   กำหนดรายละเอียดของ Pod ภายใน เช่น Container, Image, Port

ตัวอย่างไฟล์ YAML (Deployment)

deployment.yaml

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web-app
  labels:
    app: web-app
spec:
  replicas: 3
  selector:
    matchLabels:
      app: web-app
  template:
    metadata:
      labels:
        app: web-app
    spec:
      containers:
      - name: nginx
        image: nginx:1.25
        ports:
        - containerPort: 80
```

1. ประเภทของ Resource
2. ชื่อและข้อมูล
3. รายละเอียดการทำงาน
4. รายละเอียดของ Pod

YAML เป็นวิธีมาตรฐานในการกำหนด Resource ใน Kubernetes ทำให้สามารถจัดเก็บใน Git และนำกลับมาใช้งานได้อย่างง่ายดาย

ตัวอย่างไฟล์ YAML ของ Pod

Pod เป็นหน่วยพื้นฐานที่สุดใน Kubernetes โดยกำหนดผ่านไฟล์ YAML ซึ่งระบุรายละเอียด เช่น ชื่อ Pod, Image ของ Container, Port และค่าต่างๆ ที่ต้องการ

องค์ประกอบสำคัญใน YAML

1. **apiVersion**
   เวอร์ชันของ API (สำหรับ Pod ใช้ v1)
2. **kind**
   ประเภทของ Resource (เช่น Pod)
3. **metadata**
   ข้อมูลเกี่ยวกับ Pod เช่น ชื่อ และlabel
4. **spec**
   รายละเอียดการทำงานของ Pod
5. **containers**
   กำหนด Container ที่จะรันใน Pod
   เช่น Image, Port, Environment
6. **ports**
   กำหนด Port ที่ Container เปิดใช้งาน

ตัวอย่างไฟล์ YAML ของ Pod

pod-nginx.yaml

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: web-pod
  labels:
    app: web
spec:
  containers:
    - name: nginx
      image: nginx:1.25
      ports:
        - containerPort: 80
```

อธิบายตัวอย่าง

1. **apiVersion: v1**
   ใช้ API version สำหรับสร้าง Pod
2. **kind: Pod**
   ระบุว่าต้องการสร้าง Resource ประเภท Pod
3. **metadata**
   กำหนดชื่อ Pod เป็น web-pod และเพิ่ม label app=web
4. **spec**
   กำหนดรายละเอียดการทำงานของ Pod
5. **containers**
   กำหนดให้รัน Container ชื่อ nginx โดยใช้ Image nginx:1.25
6. **ports**
   เปิดใช้งาน Port 80 ภายใน Container

ไฟล์ YAML เป็นเพียงการกำหนดสิ่งที่ต้องการ (Declarative) แล้วให้ Kubernetes จัดการสร้างและรัน Pod ตามที่ระบุ

ตัวอย่างไฟล์ YAML ของ Deployment

Deployment ใช้สำหรับสร้างและจัดการกลุ่มของ Pod รวมถึงการอัปเดตแอปพลิเคชันแบบอัตโนมัติ

หน้าที่ของ Deployment



สร้างและดูแล Pod

รักษาจำนวน Pod ตามที่กำหนด



รองรับการอัปเดตอัตโนมัติ

เช่น Rolling Update



กู้คืนอัตโนมัติ

หาก Pod ล้มเหลว จะสร้างใหม่ทันที



ทำงานร่วมกับ Service

ให้ Pod เข้าถึงได้ผ่าน Service

ตัวอย่างไฟล์ YAML ของ Deployment

deployment.yaml

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web-app
  labels:
    app: web-app
spec:
  replicas: 3
  selector:
    matchLabels:
      app: web-app
  template:
    metadata:
      labels:
        app: web-app
    spec:
      containers:
      - name: nginx
        image: nginx:1.25
        ports:
        - containerPort: 80
```

1 ข้อมูลทั่วไป

2 จำนวน Pod ที่ต้องการ

3 ตัวเลือก Pod ที่จะจัดการ

4 รายละเอียดของ Pod

5 กำหนด Container

คำสั่งที่ใช้

`kubectl apply -f deployment.yaml`
สร้าง Deployment จากไฟล์ YAML

`kubectl get deployments`
ดูรายการ Deployment

`kubectl get pods`
ดู Pod ที่ถูกสร้างจาก Deployment

`kubectl describe deployment web-app`
ดูรายละเอียดของ Deployment

`kubectl delete -f deployment.yaml`
ลบ Deployment

ผลลัพธ์หลังสร้าง Deployment

```
$ kubectl get pods
```

NAME	READY	STATUS	RESTARTS	AGE
web-app-6d7f9c8d7-abc12	1/1	Running	0	2m
web-app-6d7f9c8d7-def34	1/1	Running	0	2m
web-app-6d7f9c8d7-ghi56	1/1	Running	0	2m



Deployment ช่วยให้เราจัดการแอปพลิเคชันได้ง่ายขึ้น โดยรักษาจำนวน Pod, รองรับการอัปเดตแบบไม่หยุดให้บริการ และกู้คืนอัตโนมัติเมื่อเกิดปัญหา

Service ใน Kubernetes

Service เป็นตัวกลางสำหรับการเข้าถึงกลุ่มของ Pod โดยให้ IP/ชื่อที่คงที่ และทำการกระจายทราฟฟิกไปยัง Pod ที่พร้อมใช้งาน

Service คืออะไร?

ให้ช่องทางการเข้าถึง Pod
มี IP/ชื่อที่คงที่ (Stable endpoint)

กระจายทราฟฟิก
ส่งคำขอไปยัง Pod ที่พร้อมใช้งาน
ด้วย Load Balancing (เช่น Round Robin)

แยกการเปลี่ยนแปลงของ Pod
แม้ Pod ถูกลบ/สร้างใหม่ Service
ยังเข้าถึงได้ที่เดิม

การทำงานของ Service



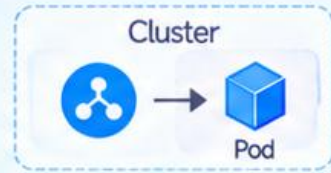
ตัวอย่างไฟล์ YAML ของ Service

service.yaml

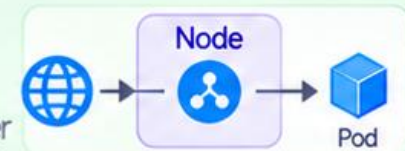
```yaml
apiVersion: v1
kind: Service
metadata:
  name: web-service
spec:
  type: ClusterIP
  selector:
    app: web-app
  ports:
    - name: http
      port: 80
      targetPort: 80
```

ประเภทของ Service

ClusterIP (ค่าเริ่มต้น)
เข้าถึงได้ภายใน Cluster เท่านั้น
ใช้สำหรับการสื่อสารระหว่าง Service



NodePort
เปิด Port บนทุก Node
เพื่อให้เข้าถึงจากภายนอก Cluster



LoadBalancer
สร้าง Load Balancer จาก Cloud Provider เพื่อให้เข้าถึงจากอินเทอร์เน็ต
ใช้ในสภาพแวดล้อม Cloud



ExternalName
เชื่อมต่อไปยังบริการภายนอก
โดยใช้ DNS name



Service ช่วยให้การเข้าถึง Pod มีความเสถียรและง่ายขึ้น โดยไม่ต้องกังวลว่าจำนวนหรือ IP ของ Pod จะเปลี่ยนแปลง

Container Image ใน Kubernetes

Kubernetes ใช้ Container Image ในการสร้าง Pod โดยดึง Image จาก Container Registry เช่น Docker Hub

Container Image คืออะไร?

แพ็กเกจของแอปพลิเคชัน
รวมโค้ด, Runtime, Dependencies และการตั้งค่า ไว้ในไฟล์เดียว

ทำงานเหมือนกันทุกที่
รันได้เหมือนกันบนเครื่องพัฒนาและใน Kubernetes

เก็บใน Container Registry
เช่น Docker Hub, GitHub Container Registry (GHCR), หรือ Private Registry

ลำดับการทำงาน



ตัวอย่างการใช้ Image ใน Pod

pod-nginx.yaml

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: nginx-pod
spec:
  containers:
    - name: nginx
      image: nginx:1.25
      ports:
        - containerPort: 80
```

pod-custom.yaml

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: web-pod
spec:
  containers:
    - name: web
      image: nginx:1.25   # ระบุ Image ที่ต้องการ
      ports:
        - containerPort: 80
```

ตัวอย่างคำสั่งใช้งาน

สร้าง Pod จาก Image

```
$ kubectl apply -f pod-nginx.yaml
```

ดู Pod ที่รันอยู่

```
$ kubectl get pods
```

ดูรายละเอียด Pod (รวม Image)

```
$ kubectl describe pod nginx-pod
```

ลบ Pod

```
$ kubectl delete -f pod-nginx.yaml
```

แหล่ง Container Registry ยอดนิยม

Docker Hub	https://hub.docker.com	Public Registry ใช้งานฟรี
GitHub Container Registry (GHCR)	https://ghcr.io	เชื่อมกับ GitHub เหมาะสำหรับโปรเจกต์ส่วนตัว/ทีม
Google Artifact Registry	https://cloud.google.com/artifact-registry	Private Registry สำหรับใช้งานในองค์กร

Container Image ช่วยให้เราสามารถรันแอปพลิเคชันใน Kubernetes ได้อย่างรวดเร็วและสอดคล้องกันทุกสภาพแวดล้อม โดยเพียงระบุชื่อ Image ในไฟล์ YAML

ตัวอย่าง: เปิดให้เข้าถึงเว็บด้วย Service

สร้าง Deployment และ Service เพื่อให้สามารถเข้าถึงเว็บแอปพลิเคชันจากภายนอกผ่าน NodePort

ขั้นตอนการทำงาน

1. **สร้าง Deployment**
   สร้าง Pod จาก Image (เช่น nginx)
2. **สร้าง Service**
   แบบ NodePort เพื่อเปิดให้เข้าถึงจากภายนอก
3. **ตรวจสอบทรัพยากร**
   ดู Pod และ Service ที่ถูกสร้าง
4. **ทดสอบการเข้าถึง**
   เข้าเว็บผ่าน IP ของ Node และ Port ที่กำหนด

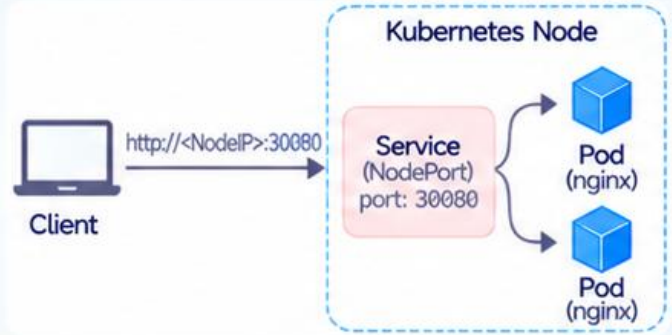
ไฟล์ Deployment (web-deployment.yaml)

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web-app
spec:
  replicas: 2
  selector:
    matchLabels:
      app: web-app
  template:
    metadata:
      labels:
        app: web-app
    spec:
      containers:
      - name: nginx
        image: nginx:1.25
        ports:
        - containerPort: 80
```

ไฟล์ Service (web-service.yaml)

```yaml
apiVersion: v1
kind: Service
metadata:
  name: web-app
spec:
  type: NodePort
  selector:
    app: web-app
  ports:
  - name: http
    port: 80
    targetPort: 80
    nodePort: 30080 # พอร์ต 30000-32767
```

ภาพรวมการเข้าถึง



คำสั่งตรวจสอบและทดสอบ

1. **สร้าง Deployment และ Service**

```
$ kubectl apply -f web-deployment.yaml
$ kubectl apply -f web-service.yaml
```

2. **ดู Pod และ Service**

```
$ kubectl get pods
$ kubectl get svc
```

3. **ทดสอบการเข้าถึงเว็บ**

```
http://<NodeIP>:30080
```

NodePort เป็นวิธีที่ง่ายที่สุดในการทดสอบการเข้าถึงจากภายนอก ในสภาพแวดล้อมจริงอาจใช้ LoadBalancer หรือ Ingress แทน

คำสั่งพื้นฐานของ kubectl

kubectl เป็นเครื่องมือสำหรับจัดการ Kubernetes โดยใช้คำสั่งจาก Command Line

คำสั่งที่ใช้บ่อย

1. **kubectl apply -f <file>**
   สร้างหรืออัปเดต Resource
2. **kubectl get <resource>**
   ดูรายการ Resource
3. **kubectl describe <resource>**
   ดูรายละเอียดของ Resource
4. **kubectl logs <pod>**
   ดู Log ของ Pod
5. **kubectl delete -f <file>**
   ลบ Resource
6. **kubectl exec -it <pod> -- <cmd>**
   เข้าภายใน Pod เพื่อรันคำสั่ง

ตัวอย่างการใช้งาน

1 สร้าง Deployment และ Service

```
$ kubectl apply -f web-deployment.yaml
$ kubectl apply -f web-service.yaml
```

2 ดู Pod ที่กำลังทำงาน

```
$ kubectl get pods
```

NAME	READY	STATUS	RESTARTS	AGE
web-app-6d7f9c8d7-abc12	1/1	Running	0	2m

3 ดูรายละเอียดของ Pod

```
$ kubectl describe pod web-app-6d7f9c8d7-abc12
Name:             web-app-6d7f9c8d7-abc12
Namespace:        default
Node:             minikube/192.168.49.2
...
```

4 ดู Log ของ Pod

```
$ kubectl logs web-app-6d7f9c8d7-abc12
/docker-entrypoint.sh: Configuration complete
2024/09/12 10:15:23 [notice] start worker processes
```

5 ลบ Deployment และ Service

```
$ kubectl delete -f web-deployment.yaml
$ kubectl delete -f web-service.yaml
```

6 เข้าภายใน Pod เพื่อรันคำสั่ง

```
$ kubectl exec -it web-app-6d7f9c8d7-abc12 -- /bin/sh
/ # ls
bin   etc   usr   var
/ # exit
```

คำสั่งเสริมที่ควรรู้

`kubectl get all`	ดูทุก Resource ใน Namespace
`kubectl get nodes`	ดูรายการ Node ใน Cluster
`kubectl get svc`	ดูรายการ Service
`kubectl get deploy`	ดูรายการ Deployment
`kubectl top pods`	ดูการใช้งาน CPU/Memory (ต้องติดตั้ง metrics)
`kubectl version`	ดูเวอร์ชันของ kubectl และ Cluster

kubectl สามารถจัดการ Resource ได้หลายประเภท เช่น Pod, Deployment, Service, ConfigMap, Secret และอื่นๆ โดยใช้ไฟล์ YAML

Workshop: สร้างเว็บ 2 อัน + Database 1 อัน บน Kubernetes

สร้างเว็บแอปพลิเคชัน 2 ตัว (web1, web2) และฐานข้อมูล 1 ตัว (MySQL) แล้ว Deploy ด้วย kubectl

1 เตรียมไฟล์แอปพลิเคชัน

สร้างโฟลเดอร์โปรเจกต์ตามโครงสร้างนี้

```
k8s-workshop/
├── web1/
│   ├── Dockerfile
│   └── index.html
└── web2/
    ├── Dockerfile
    └── index.html
```

2 เขียนเว็บแอป 2 อัน (ต่างกันคนละสี)

web1/index.html

```html
<html>
<head><title>Web1</title></head>
<body style="background:#e3f2fd;
   <h1>Web App 1</h1>
   <p>Connected to MySQL</p>
</body>
</html>
```

web2/index.html

```html
<html>
<head><title>Web2</title></head>
<body style="background:#fce4ec;
   <h1>Web App 2</h1>
   <p>Connected to MySQL</p>
</body>
</html>
```

3 เขียน Dockerfile (ทั้ง 2 เว็บใช้เหมือนกัน)

web1/Dockerfile และ web2/Dockerfile

```
FROM nginx:alpine
COPY index.html /usr/share/nginx/html/index.html
EXPOSE 80
```

4 สร้าง Image และ Push (ถ้าใช้ Docker Hub)

```
# เข้าไปที่โฟลเดอร์ web1
cd web1
docker build -t <yourname>/web1:v1 .
docker push <yourname>/web1:v1

# ทำเช่นเดียวกันสำหรับ web2
cd ../web2
docker build -t <yourname>/web2:v1 .
docker push <yourname>/web2:v1
```

5 สร้างไฟล์ YAML สำหรับ Database (MySQL)

mysql.yaml

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: mysql
spec:
  replicas: 1
  selector:
    matchLabels:
      app: mysql
  template:
    metadata:
      labels:
        app: mysql
    spec:
      containers:
        - name: mysql
          image: mysql:8.0
          env:
            - name: MYSQL_ROOT_PASSWORD
              value: mypassword
          ports:
            - containerPort: 3306
```

6 สร้างไฟล์ YAML สำหรับ Web1 และ Web2

web1.yaml

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web1
spec:
  replicas: 1
  selector:
    matchLabels:
      app: web1
  template:
    metadata:
      labels:
        app: web1
    spec:
      containers:
        - name: web1
          image: <yourname>/web1:v1
          ports:
            - containerPort: 80
```

web2.yaml

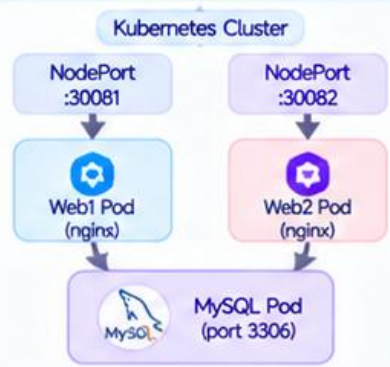
```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web2
spec:
  replicas: 1
  selector:
    matchLabels:
      app: web2
  template:
    metadata:
      labels:
        app: web2
    spec:
      containers:
        - name: web2
          image: <yourname>/web2:v1
          ports:
            - containerPort: 80
```

7 Deploy และทดสอบการใช้งาน

```
# สร้างทั้งหมด
kubectl apply -f mysql.yaml
kubectl apply -f web1.yaml
kubectl apply -f web2.yaml

# ดูสถานะ
kubectl get pods
kubectl get svc

# ทดสอบการเข้าถึง (ผ่าน NodePort)
# ใช้ IP ของ Node ที่ให้มาในสภาพแวดล้อม
http://<NodeIP>:30081  (Web1)
http://<NodeIP>:30082  (Web2)
```

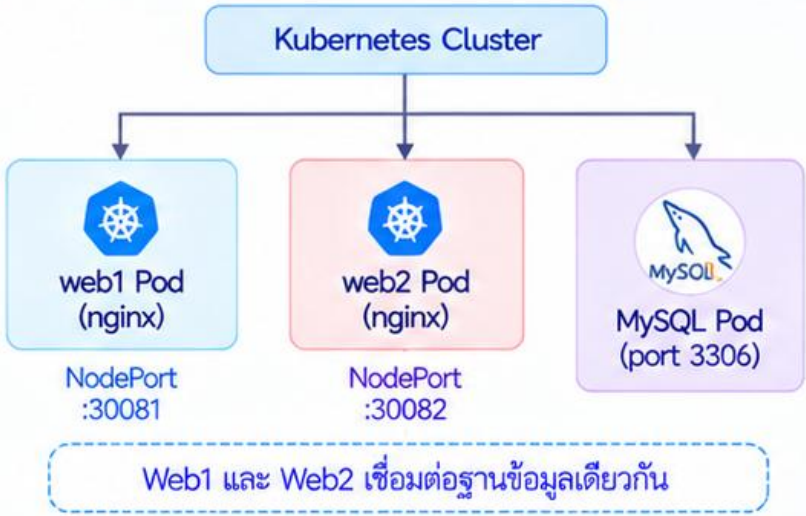


Workshop นี้ช่วยให้เห็นการสร้างแอปพลิเคชันหลายตัว ทำงานร่วมกับฐานข้อมูลเดียวกัน และเข้าใช้งานได้จากภายนอกผ่าน NodePort

ผลลัพธ์ของ Workshop

เมื่อทำตามขั้นตอนแล้ว จะได้เว็บแอปพลิเคชัน 2 ตัว ที่เชื่อมต่อกับฐานข้อมูล 1 ตัว บน Kubernetes

สถาปัตยกรรมที่ได้



เข้าถึงเว็บแอปพลิเคชัน

Web1 (NodePort 30081)

```
http://<NodeIP>:30081
```

- แสดงหน้าเว็บ Web1
- เชื่อมต่อและดึงข้อมูลจาก MySQL

Web2 (NodePort 30082)

```
http://<NodeIP>:30082
```

- แสดงหน้าเว็บ Web2
- เชื่อมต่อและดึงข้อมูลจาก MySQL (ใช้ฐานข้อมูลเดียวกัน)

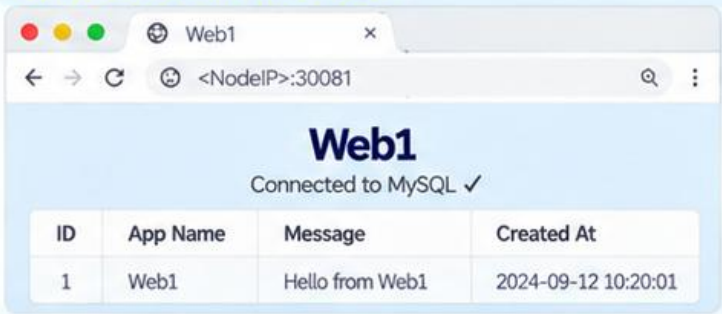
ตรวจสอบฐานข้อมูล

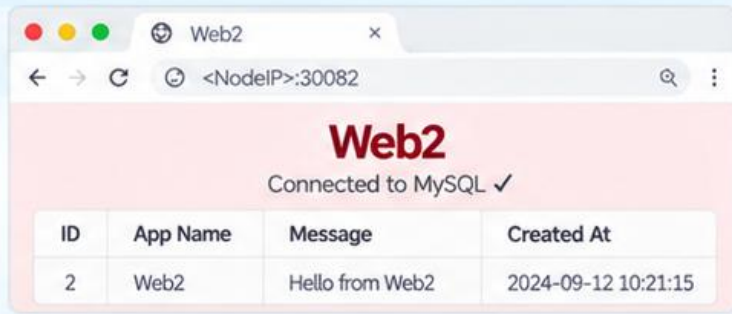
```
kubectl exec -it mysql-xxx -- mysql -u root -p
```

```
USE appdb;
SELECT * FROM visitors;
```

```
+----+----------+--------------+---------------------+
| id | app_name | message      | created_at          |
+----+----------+--------------+---------------------+
|  1 | Web1     | Hello from Web1 | 2024-09-12 10:20:01 |
|  2 | Web2     | Hello from Web2 | 2024-09-12 10:21:15 |
+----+----------+--------------+---------------------+
```

ตัวอย่างหน้าเว็บที่ได้





สิ่งที่นักศึกษาได้เรียนรู้จาก Workshop นี้

1. สร้างและจัดการ Container Image ด้วย Docker
2. เขียนไฟล์ YAML สำหรับ Deployment, Service และ Database
3. Deploy แอปพลิเคชันหลายตัวบน Kubernetes
4. เชื่อมต่อแอปพลิเคชันกับฐานข้อมูลภายใน Cluster
5. ทดสอบการทำงานและตรวจสอบข้อมูล

Workshop นี้เป็นตัวอย่างการประยุกต์ใช้ความรู้ Docker และ Kubernetes ในการรันแอปพลิเคชันจริงที่มีหลายบริการ (Multi-Service Application)

ขั้นตอนที่ 1: เตรียมไฟล์ Workshop

ดาวน์โหลดและตรวจสอบไฟล์สำหรับสร้างแอปพลิเคชัน Web 2 ตัว + Database 1 ตัว บน Kubernetes

1 ดาวน์โหลดไฟล์ตัวอย่าง

ดาวน์โหลดไฟล์จากเครื่องของคุณ
(หรือจากลิงก์ที่ผู้สอนให้)

bash

```
unzip k8s-web2-mysql-workshop.zip
cd k8s-web2-mysql-workshop
```

2 โครงสร้างไฟล์

ไฟล์ที่ได้จะมีโครงสร้างดังนี้

```
k8s-web2-mysql-workshop/
├── app/
│   ├── app.py
│   ├── requirements.txt
│   └── Dockerfile
├── k8s/
│   ├── mysql.yaml
│   └── web.yaml
└── README.md
```

3 โค้ดแอปพลิเคชัน (app/app.py)

ตัวอย่างโค้ดบางส่วน: แอปพลิเคชัน Flask ที่เชื่อมต่อ MySQL

python

```python
import os
import mysql.connector
from flask import Flask, render_template_string

app = Flask(__name__)
# รับค่าจาก Environment Variable
DB_HOST = os.getenv("DB_HOST", "mysql-service")
DB_USER = os.getenv("DB_USER", "root")
DB_PASSWORD = os.getenv("DB_PASSWORD", "cloud123")
DB_NAME = os.getenv("DB_NAME", "appdb")
APP_NAME = os.getenv("APP_NAME", "Web")

@app.route("/")
def index():
    # เชื่อมต่อ MySQL
    conn = mysql.connector.connect(
        host=DB_HOST, user=DB_USER,
        password=DB_PASSWORD, database=DB_NAME
    )
    cursor = conn.cursor()
    # บันทึกข้อมูลการเข้ามาใช้งาน
    cursor.execute(
        "INSERT INTO visitors (app_name, message) "
        "VALUES (%s, %s)", (APP_NAME, f"Hello from {APP_NAME}")
    )
    conn.commit()
    # อ่านข้อมูลจากฐานข้อมูล
    cursor.execute("SELECT id, app_name, message, created_at "
                   "FROM visitors ORDER BY id DESC LIMIT 10")
    rows = cursor.fetchall()
    cursor.close()
    conn.close()
# แสดงผลหน้าเว็บ
html = render_template_string("""
<h1>{{ app_name }}</h1>
<p>Connected to MySQL ✓</p>
<form method="post" action="/add">
  <input name="message" placeholder="Type a message" required>
  <button type="submit">Add Message</button>
</form>
<table border="1">
  <tr><th>ID</th><th>App Name</th><th>Message</th><th>Created At</th></tr>
  {% for row in rows %}
    <tr><td>{{ row[0] }}</td><td>{{ row[1] }}</td>
      <td>{{ row[2] }}</td><td>{{ row[3] }}</td></tr>
  {% endfor %}
</table>
""", app_name=APP_NAME, rows=rows)
return html
```

4 Dockerfile (ใช้ร่วมกันทั้ง Web1 และ Web2)

สร้าง Docker Image จากแอปพลิเคชัน Python Flask

dockerfile

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY app.py .
EXPOSE 5000
CMD ["python", "app.py"]
```

5 requirements.txt

ไลบรารีที่จำเป็นสำหรับแอปพลิเคชัน

text

```
Flask==3.0.0
mysql-connector-python==8.2.0
```

6 ไฟล์ Kubernetes YAML

มี 2 ไฟล์ สำหรับ MySQL และ Web Application

k8s/mysql.yaml
Deployment และ Service ของ MySQL

yaml

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: mysql
... (ดูรายละเอียดในไฟล์)
```

k8s/web.yaml
Deployment และ Service ของ Web1 และ Web2

yaml

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web1
... (มีทั้ง web1, web2 และ Service)
```

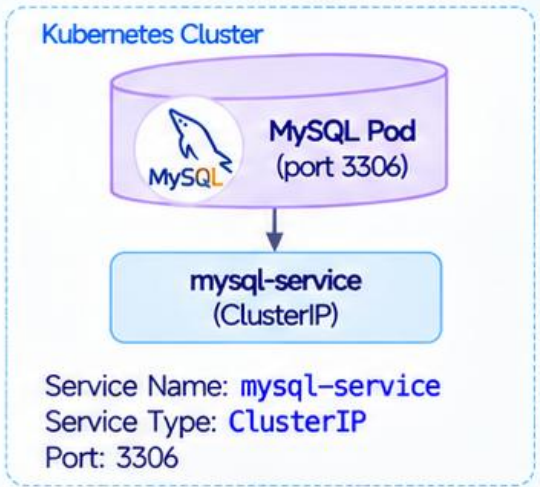
เมื่อเตรียมไฟล์เรียบร้อยแล้ว ในขั้นตอนถัดไปเราจะเริ่ม Deploy ฐานข้อมูล MySQL ครับ

ขั้นตอนที่ 2: Deploy Database (MySQL)

สร้าง MySQL Deployment และ Service สำหรับให้ Web1 และ Web2 เชื่อมต่อ

1 ภาพรวม

สร้าง MySQL Deployment และ Service ชื่อ mysql-service เพื่อให้ Web1 และ Web2 เชื่อมต่อผ่านชื่อ Service นี้



2 ดูไฟล์ mysql.yaml

ไฟล์ k8s/mysql.yaml จะมี 2 ส่วน คือ

- MySQL Deployment (1 replica)
- MySQL Service (ClusterIP)

```
k8s-web2-mysql-workshop/
├── app/
│   ├── app.py
│   ├── requirements.txt
│   └── Dockerfile
├── k8s/
│   ├── mysql.yaml
│   └── web.yaml
└── README.md
```

3 ตัวอย่างไฟล์ k8s/mysql.yaml

Deployment และ Service ของ MySQL

MySQL Deployment

k8s/mysql.yaml

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: mysql
spec:
  replicas: 1
  selector:
    matchLabels:
      app: mysql
  template:
    metadata:
      labels:
        app: mysql
    spec:
      containers:
      - name: mysql
        image: mysql:8.0
        ports:
        - containerPort: 3306
        env:
        - name: MYSQL_ROOT_PASSWORD
          value: cloud123
        - name: MYSQL_DATABASE
          value: appdb
        volumeMounts:
        - name: mysql-data
          mountPath: /var/lib/mysql
      volumes:
      - name: mysql-data
        emptyDir: {}   # ใช้ในสภาพแวดล้อมทดลอง
```

MySQL Service

k8s/mysql.yaml

```yaml
apiVersion: v1
kind: Service
metadata:
  name: mysql-service
spec:
  type: ClusterIP
  selector:
    app: mysql
  ports:
  - port: 3306
    targetPort: 3306
```

4 Deploy MySQL

ใช้คำสั่ง kubectl สร้าง Deployment และ Service

bash

```
kubectl apply -f k8s/mysql.yaml
```

```
deployment.apps/mysql created
service/mysql-service created
```

5 ตรวจสอบสถานะ

ตรวจสอบว่า Pod และ Service ของ MySQL ทำงานอยู่

bash

```
kubectl get pods -l app=mysql
```

NAME	READY	STATUS	RESTARTS	AGE
mysql-6d7f9c8d7-abc12	1/1	Running	0	1m

bash

```
kubectl get svc mysql-service
```

NAME	TYPE	CLUSTER-IP	PORT(S)	AGE
mysql-service	ClusterIP	10.96.123.45	3306/TCP	1m

สิ่งที่ควรสังเกต

- Service ชื่อ mysql-service ใช้สำหรับให้ Web1 และ Web2 เชื่อมต่อ
- เป็น ClusterIP (ใช้ได้ภายใน Cluster เท่านั้น)
- Web1 และ Web2 จะเชื่อมต่อที่ host "mysql-service" และ port 3306
- ข้อมูล MySQL จะถูกเก็บใน emptyDir (เหมาะสำหรับการทดลอง)

ขั้นตอนที่ 3: Deploy Web1 และ Web2

สร้าง Web Application 2 ตัว ที่เชื่อมต่อ MySQL และเปิดให้เข้าถึงด้วย NodePort

1 ดูไฟล์ k8s/web.yaml

ไฟล์ k8s/web.yaml มี Deployment และ Service สำหรับ Web1 และ Web2

k8s/web.yaml

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web1
spec:
  replicas: 1
  selector:
    matchLabels:
      app: web1
  template:
    metadata:
      labels:
        app: web1
    spec:
      containers:
      - name: web-app
        image: web-app:1.0
        ports:
        - containerPort: 5000
        env:
          - name: APP_NAME
            value: "Web1"
          - name: DB_HOST
            value: "mysql-service"
          - name: DB_USER
            value: "root"
          - name: DB_PASSWORD
            value: "cloud123"
          - name: DB_NAME
            value: "appdb"
```

(ส่วน Web2 เหมือนกัน แต่เปลี่ยนชื่อเป็น web2 และ APP_NAME=Web2)

2 Deploy Web1 และ Web2

ใช้คำสั่ง kubectl สร้าง Deployment และ Service

bash

```
kubectl apply -f k8s/web.yaml
```

```
deployment.apps/web1 created
service/web1-service created
deployment.apps/web2 created
service/web2-service created
```

3 ตรวจสอบสถานะ

ตรวจสอบว่า Pod และ Service ของ Web1 และ Web2 ทำงานอยู่

bash

```
kubectl get pods
```

NAME	READY	STATUS	RESTARTS	AGE
web1-5d7f9c8d7-abc12	1/1	Running	0	1m
web2-7e8f9c8d7-def34	1/1	Running	0	1m

bash

```
kubectl get svc
```

NAME	TYPE	CLUSTER-IP	PORT(S)	AGE
mysql-service	ClusterIP	10.96.123.45	3306/TCP	3m
web1-service	NodePort	10.96.234.56	5000:30081/TCP	1m
web2-service	NodePort	10.96.234.57	5000:30082/TCP	1m

4 ทดสอบการเข้าถึง Web1 และ Web2

ใช้ NodePort ที่ได้จาก Service เปิดเว็บในเบราว์เซอร์

Web1 (NodePort 30081)



ID	App Name	Message	Created At
1	Web1	Hello from Web1	2024-09-12 10:20:01
2	Web2	Hello from Web2	2024-09-12 10:24:35

Web2 (NodePort 30082)



ID	App Name	Message	Created At
1	Web1	Hello from Web1	2024-09-12 10:20:01
2	Web2	Hello from Web2	2024-09-12 10:24:35

5 ตรวจสอบข้อมูลใน MySQL

เชื่อมต่อไปยัง MySQL Pod และตรวจสอบข้อมูลในตาราง

bash

```
kubectl exec -it mysql-xxx -- mysql -u root -p
USE appdb;
SELECT * FROM visitors;
```

```
mysql> SELECT * FROM visitors;
+----+----------+----------------------+---------------------+
| id | app_name | message              | created_at          |
+----+----------+----------------------+---------------------+
| 1  | Web1     | Hello from Web1      | 2024-09-12 10:20:01 |
| 2  | Web2     | Hello from Web2      | 2024-09-12 10:24:35 |
| 3  | Web1     | Hello from Web1      | 2024-09-12 10:25:01 |
+----+----------+----------------------+---------------------+
```

สิ่งที่ควรสังเกต

- Web1 และ Web2 ใช้ Image เดียวกัน แต่กำหนดค่า APP_NAME ต่างกัน
- Web เชื่อมต่อ MySQL ผ่าน Service ชื่อ "mysql-service"
- เข้าถึง Web ได้ผ่าน NodePort 30081 (Web1) และ 30082 (Web2)
- ข้อมูลจากทั้งสองเว็บถูกบันทึกใน Database เดียวกัน

Thank you

Q / A

ขอบคุณสำหรับการเรียนรู้ร่วมกันในวันนี้ครับ



